

Série des exercices N°4

Exercice 1: Réaliser une classe *point* permettant de manipuler un point d'un plan :

- un constructeur recevant en arguments les coordonnées (x et y); Les coordonnées du point seront des membres données privées.
- une fonction membre *deplace* effectuant une translation définie par ses deux arguments (*dx* , *dy*)
- une fonction membre *affiche* permettant d'afficher les coordonnées cartésiennes du point.
- une fonction *homothetie* qui effectue une homothétie dont le paramètre m est fourni en argument; cette méthode multiplie les coordonnées de l'objet par m.
- une fonction *PointSymmetriqueParRapportAO* qui retourne le point symétrique par rapport à l'origine O.
- une fonction *PointSymmetriqueParRapportAxeDesAbscisses* qui retourne le point symétrique par rapport à l'axe des abscisses.
- Ajouter à la classe Point un attribut statique NbPoints qui représentera le nombre d'objets instanciés de cette classe. Ainsi il sera incrémenté à chaque appel du constructeur `__init__` et il sera décrémenté à chaque appel du destructeur `__del__`

Exercice 2:

Écrire les méthodes demandées pour manipuler des fractions.
Chaque question correspond à une méthode de la classe Fraction.

1) Constructeur:

Écrire le constructeur `__init__` de la classe Fraction qui :

- prend en paramètres le numérateur n et le dénominateur d ;
- vérifie que le dénominateur n'est pas nul (`assert d != 0`) ;
- met le signe sur le numérateur si nécessaire (d négatif) ;
- initialise `self.num` et `self.den` et simplifie la fraction.

2) Affichage:

Écrire la méthode `__str__` qui renvoie :

'num' si le dénominateur vaut 1 et 'num/den' sinon.

3) Comparaison:

Écrire la méthode `__eq__` pour comparer deux fractions self et other et retourner True si elles sont égales, sinon False.

4) Calcul du PGCD:

Module: Python pour les sciences de données

Écrire une méthode `_pgcd(self, a, b)` qui calcule le Plus Grand Commun Diviseur de deux entiers `a` et `b`.

5) Simplification en place:

Écrire la méthode `_simplifie_en_place` qui réduit la fraction `self.num/self.den` à sa forme la plus simple en utilisant `_pgcd`.

6) Simplification avec nouvelle fraction:

Écrire la méthode `simplifie` qui retourne une nouvelle fraction simplifiée sans modifier la fraction originale.

7) Addition:

Écrire la méthode `__add__` pour additionner `self` et `other` et retourner une nouvelle fraction simplifiée.

8) Soustraction:

Écrire la méthode `__sub__` pour soustraire `other` de `self` et retourner une fraction simplifiée.

9) Multiplication:

Écrire la méthode `__mul__` pour multiplier deux fractions et retourner une fraction simplifiée.

10) Division:

Écrire la méthode `__truediv__` pour diviser `self` par `other` :

- vérifier que `other.num` n'est pas nul (`assert`)
- retourner une fraction simplifiée.

11) Conversion en float:

Écrire une méthode `to_float` qui convertit la fraction en nombre flottant.

12) Comparaison:

Écrire la méthode `__lt__` pour permettre de comparer deux fractions avec `<`.

Exercice 3: Classe Etudiant (version 4 matières)

On souhaite créer une classe Etudiant permettant de gérer les notes de 4 matières :

- Python pour Data Science
- Big Data
- Machine Learning

- Statistiques avancées

1) Créez une classe Etudiant dont le constructeur `__init__` reçoit :

- le nom et le prénom de l'étudiant,
- ainsi que ses quatre notes dans les matières citées ci-dessus.

Les attributs de la classe seront donc : `nom`, `prenom`, `note_py`, `note_BD`, `note_ML`, `note_SA`.

2) Méthode d'affichage :

- Ajoutez une méthode `__str__(self)` qui renvoie une phrase du type :
HASSANI Ismail a obtenu 18 en Python, 18 en Big Data, 15 en ML, 16 en Statistiques avancées.
- Testez cette méthode en créant un étudiant et en l'affichant avec `print()`.

3) Calcul de la moyenne :

- Ajoutez une méthode `moyenne(self)` qui calcule et renvoie la moyenne des 4 notes.
- Testez la méthode sur plusieurs étudiants.

4) Comparaison entre étudiants :

Ajoutez une méthode `__lt__(self, other)` qui renvoie `True` si la moyenne de `self` est strictement inférieure à celle de `other`.

👉 Cela permettra d'utiliser l'opérateur `<` entre deux étudiants.

5) Création et affichage d'étudiants :

Créez les instances suivantes :

- `E1 = Etudiant("HASSANI", "Ismail", 18, 18, 15, 16)`
`E2 = Etudiant("BOURASS", "MOUAD", 17, 15, 16, 14)`
`E3 = Etudiant("ALAMI", "Ghita", 14, 19, 18, 17)`
`E4 = Etudiant("MOUMNI", "Latifa", 15, 17, 19, 18)`

`print(E1) ; print(E2) ; print(E3) ; print(E4)`

6) Mentions :

Ajoutez une méthode `mention(self)` qui renvoie une appréciation selon la moyenne :

- ≥ 16 : Excellent
- ≥ 14 : Bien
- ≥ 12 : Assez bien
- ≥ 10 : Passable
- < 10 : Insuffisant

Exemple :

```
print(E1.mention()) # affiche "Excellent"  
print(E2.mention()) # affiche "Bien"
```

Exercice 4: un petit système pour gérer des personnes, des étudiants et des candidats à un concours.

Classe Personne

- Attributs : nom, prenom
- Constructeur avec valeurs par défaut 'anonyme'
- Méthodes : getNom(), setNom(nom), getPrenom(), setPrenom(prenom), affiche()

Classe Etudiant (hérite de Personne)

- Attributs supplémentaires : CNE, filiere
- Constructeur avec valeurs par défaut
- Accesseurs et mutateurs pour CNE et filiere
- Redéfinir affiche() pour afficher aussi le CNE et la filière

Classe CandidatPourUnConcours (hérite de Etudiant)

- Attributs supplémentaires : numeroExam, moyenneGenerale
- Constructeur avec valeurs par défaut
- Accesseurs et mutateurs pour numeroExam et moyenneGenerale
- Redéfinir affiche() pour afficher toutes les informations

Polymorphisme : Écrire une fonction afficheSelonType(personne) qui prend un objet et appelle sa méthode affiche()

Travail à faire :

- 1). Implémentez les trois classes selon les consignes ci-dessus.
- 2). Dans le main :
 - Créez :
 - une Personne 'HASSANI', 'Ismail'
 - un Etudiant 'ALAMI', 'Ghita', 5678, 'Analyse Mathématiques'
 - un CandidatPourUnConcours 'MOUMNI', 'Latifa', 3456, 'SMA', 5, 14.5
 - Utilisez afficheSelonType() pour afficher les informations de chaque objet.
- 3). Modifiez la filière de l'étudiant avec son mutateur et affichez les nouvelles informations.
- 4). redéfinissez __str__ pour chaque classe afin de pouvoir utiliser print(obj) directement.

Exemple d'exécution attendu :

Nom : HASSANI , Prénom : Ismail

Nom : ALAMI , Prénom : Ghita , CNE : 5678 , Filière : Intelligence artificielle

Nom : MOUMNI , Prénom : Latifa , CNE : 3456 , Filière : DSBD , Numéro d'examen :
5, Moyenne Générale : 14.5

Après modification de la filière de l'étudiant :

Nom : ALAMI , Prénom : Ghita , CNE : 5678 , Filière : Informatique